explain the project for your AI-Powered Code Reviewer using MERN + Google Generative AI project.

🚀 Project Overview (Simple & Professional)

Project Title: AI-Powered Code Reviewer using MERN Stack
Tech Stack: MongoDB, Express.js, React.js, Node.js, Google Generative AI API, Axios, Prism.js, Markdown Renderer, Vite

✅ Overview

This project is a full-stack web application that allows users to submit code and get AI-generated code reviews, suggestions, optimizations, and best-practice recommendations. It uses:

- React (Vite) for frontend UI
- Prism.js for syntax highlighting
- Axios for API communication
- Node.js + Express as backend server
- Google Generative AI API to analyze and review code
- Markdown rendering to show AI output in a clean, formatted way
- CORS setup for secure communication

The AI model analyzes user-submitted code and generates:
✔ Code improvements
✔ Bug detection
✔ Optimization suggestions
✔ Clean coding practices

This project helps in understanding AI integration, frontend-backend communication, and building intelligent developer tools.

🎤 How to Explain This Project in Interviews (Your Script)

Short Version (30 seconds)

"My project is an AI-Powered Code Reviewer built with MERN stack. The user pastes a code snippet in the frontend, which sends a request to a Node.js backend. The backend communicates with Google Generative AI API to analyze the code and return suggestions. I used Prism.js for syntax highlighting and Markdown rendering for clean output. The project demonstrates my ability to integrate AI with a full-stack app."

Long Version (1–2 minutes)

"I developed an AI-powered Code Reviewer using the MERN stack.
The frontend is built using React with Vite for faster setup and better performance. Users can paste their code in a syntax-highlighted editor powered by Prism.js. When they click Review, the code is sent to the backend via Axios.

The backend is built in Node.js + Express. I added CORS support and created a POST API

I used Prism.js for syntax highlighting and Markdown rendering for clean output. The project demonstrates my ability to integrate AI with a full-stack app."

Long Version (1–2 minutes)

"I developed an AI-powered Code Reviewer using the MERN stack.
The frontend is built using React with Vite for faster setup and better performance. Users can paste their code in a syntax-highlighted editor powered by Prism.js. When they click Review, the code is sent to the backend via Axios.

The backend is built in Node.js + Express. I added CORS support and created a POST API to send the request to Google Generative AI API. The model analyzes the code and returns structured feedback like bug fixes, optimization suggestions, best practices, complexity reduction, and comments for improvement.

The response is rendered in the frontend using a Markdown renderer for neat formatting. This project taught me how to integrate an AI model into a MERN application, handle API responses, manage secure keys, and improve UI/UX for developers."

❓ Interview Questions & Answers (Best for This Project)

---

1. What is the purpose of this project?

Answer:
The purpose is to build a tool that reviews code automatically using AI. It provides suggestions, finds issues, and improves code quality. It demonstrates full-stack development with AI integration.

---

2. Why did you use Google Generative AI?

Answer:
Google Generative AI (Gemini) is powerful and has excellent support for text generation and code understanding. It can analyze programming patterns, detect issues, and provide high-quality suggestions, which makes it ideal for building an AI code reviewer.

---

3. Explain the architecture of your project.

Answer:

- Frontend (React + Vite): Code input area, syntax highlighting using Prism.js, and result display using Markdown.
- Backend (Node + Express): REST API to receive code and send it to the AI.
- AI Layer: Google Generative AI API processes the input and returns a detailed review.
- Communication: Axios on frontend → POST request → Node backend → AI API → Response back to frontend.

high-quality suggestions, which makes it ideal for building an AI code reviewer.

---

3. Explain the architecture of your project.

Answer:

- Frontend (React + Vite): Code input area, syntax highlighting using Prism.js, and result display using Markdown.
- Backend (Node + Express): REST API to receive code and send it to the AI.
- AI Layer: Google Generative AI API processes the input and returns a detailed review.
- Communication: Axios on frontend → POST request → Node backend → AI API → Response back to frontend.

---

4. Why did you choose MERN stack?

Answer:
MERN provides seamless JS-based development.
React for UI, Node+Express for backend, MongoDB for saving user history (optional), making it easy to scale and maintain.

---

5. How does the AI review feature work?

Answer:

1. User enters code.
2. Frontend sends POST request with code.
3. Backend formats prompt and sends to Google Gen AI.
4. AI returns suggestions.
5. Frontend displays response with Markdown.

---

6. What is the role of Prism.js in your project?

Answer:
Prism.js provides syntax highlighting. It improves readability and makes the code review interface look like a real IDE.

---

7. How did you handle CORS?

Answer:
I enabled CORS in the Express backend using the CORS middleware to allow controlled communication between frontend and backend running on different ports.

---

8. Why did you use POST instead of GET for AI requests?

Answer:
POST is more secure and suitable for sending larger data, like long code snippets. GET has URL length limits and is not meant for sensitive content.

I enabled CORS in the Express backend using the CORS middleware to allow controlled communication between frontend and backend running on different ports.

---

8. Why did you use POST instead of GET for AI requests?

Answer:
POST is more secure and suitable for sending larger data, like long code snippets. GET has URL length limits and is not meant for sensitive content.

---

9. How do you render AI response cleanly?

Answer:
Using a Markdown renderer, which formats headings, bullet points, code blocks, etc., directly from the AI output.

---

10. What challenges did you face?

Answer (choose your version):

- Handling large AI responses.
- Formatting Markdown properly.
- Managing API keys securely.
- Improving UI for code readability.
- Ensuring stable communication between frontend and backend.

---

11. How is the prompt structured for the AI?

Answer:
The backend sends a structured prompt like:
"Review this code. Identify issues, bugs, improvements, best practices, and rewrite optimized code."

---

12. How would you scale this project?

Answer:

- Add authentication
- Save user code reviews in MongoDB
- Add multiple programming language support
- Add real-time linting
- Deploy using Vercel + Render

🧠 Additional Smart Points to Say in Interview

- "I converted raw AI text into clean Markdown for better readability."
- "I optimized the backend to use POST for structured requests."
- "I refactored the project into reusable components."

Give me all positive interview questions and answers for this project by hr such as technical, behaviour , soft skills and scenarios based

Below is a complete, well-structured list of positive interview questions and answers for your AI-Powered Code Reviewer (MERN + Google GenAI) project.
These cover HR, Technical, Behavioral, Soft Skills, Scenario-based, and Project-specific questions.

Use them directly in interviews — they highlight confidence, ownership, and clarity.

✅ 1. Project-Specific Technical Questions (With Positive Answers)

Q1: Can you explain your AI-Powered Code Reviewer project in simple terms?

A:
Yes. I built a full-stack web application using the MERN stack that reviews developer code using Google Generative AI.
Users paste their code in a syntax-highlighted editor, and the backend sends it to the AI model.
The AI analyzes the code and returns suggestions, improvements, and best practices.
On the frontend, I show the formatted review using Markdown.

Q2: Why did you choose the MERN stack for this project?

A:
MERN allows full JavaScript development across frontend and backend, which makes development faster and smoother.
React gives an excellent UI experience, Express + Node are lightweight for APIs, and MongoDB can store user history if needed.

Q3: Which AI model did you use and why?

A:
I used Google Generative AI (Gemini).
It's strong in understanding both natural language and code.
It provides detailed analysis, identifies bugs, and explains improvements clearly — perfect for a code reviewer tool.

Q4: What was the biggest technical challenge you solved?

A:
Managing clean AI output.
The raw AI text had irregular formatting, so I added a Markdown renderer to show headings, bullets, and code blocks cleanly.
This improved the readability and UX significantly.

Q5: How does the backend communicate with the AI model?

A:
The backend sends a POST request containing the code snippet and a structured prompt

for a code reviewer tool.

Q4: What was the biggest technical challenge you solved?

A:
Managing clean AI output.
The raw AI text had irregular formatting, so I added a Markdown renderer to show headings, bullets, and code blocks cleanly.
This improved the readability and UX significantly.

Q5: How does the backend communicate with the AI model?

A:
The backend sends a POST request containing the code snippet and a structured prompt to the Gemini API.
The API returns a code review response, which I forward to the frontend.

Q6: Why did you use POST instead of GET?

A:
POST handles large payloads like long code snippets and keeps data secure.
GET has limitations and exposes data in URLs.

Q7: What libraries did you use in the frontend and why?

A:

- Prism.js → for syntax highlighting
- Axios → for API calls
- Markdown Renderer → to display AI output cleanly
- Vite → for faster React development

Q8: How do you handle CORS issues?

A:
I enabled the CORS middleware in Express and allowed only the frontend origin.
This keeps communication secure.

Q9: Can your project support multiple programming languages?

A:
Yes. Google GenAI supports multiple languages.
Right now, the UI supports plain text input, but it's easy to extend Prism.js to highlight multiple languages.

Q10: How would you scale this project?

A:

- Add user authentication
- Store user code history
- Add real-time linting
- Add multiple languages support
- Deploy on Vercel + Render
- Improve analytics and caching

multiple languages.

Q10: How would you scale this project?

A:

- Add user authentication
- Store user code history
- Add real-time linting
- Add multiple languages support
- Deploy on Vercel + Render
- Improve analytics and caching

✅ 2. Behavioural Questions (Positive Answers)

Q11: How do you handle challenging tasks in a project?

A:
I break the task into smaller steps, research efficiently, and test in iterations.
If needed, I communicate early with the team instead of waiting until the end.

Q12: How do you learn new technologies like Google GenAI?

A:
I follow documentation, build small prototypes first, and experiment.
Hands-on practice helps me learn faster.

Q13: Describe a situation where you took initiative.

A:
In this project, the AI output was messy initially.
Without being asked, I researched and implemented Markdown rendering to make the output readable.
This significantly improved the UX.

Q14: How do you handle feedback?

A:
I treat feedback as a chance to improve.
I listen carefully, ask questions if needed, and apply the suggestions.

Q15: What motivates you to work on projects like this?

A:
Building something that solves real developer problems motivates me.
I enjoy seeing ideas turn into functional solutions.

✅ 3. Soft Skills Questions (Positive Answers)

Q16: How do you manage your time while working on projects?

A:

Building something that solves real developer problems motivates me.
I enjoy seeing ideas turn into functional solutions.

✅ 3. Soft Skills Questions (Positive Answers)

Q16: How do you manage your time while working on projects?

A:
I plan features in small milestones, prioritize tasks, and track progress daily.
This helps avoid last-minute rush and keeps development smooth.

Q17: How do you communicate technical ideas to non-technical people?

A:
I simplify the explanation using examples or analogies.
I avoid jargon and focus on the “why” and “benefits.”

Q18: Describe your teamwork style.

A:
I’m collaborative.
I share ideas, help others, and ensure everyone is aligned.
I like keeping communication open and clear.

Q19: How do you stay updated with new technologies?

A:
I follow official documentation, blogs, YouTube channels, and build mini projects to apply what I learn.

Q20: What is your approach to debugging?

A:
I reproduce the issue, isolate the cause, check logs, and fix step-by-step.
I stay calm and logical during debugging.

✅ 4. HR General Questions (Positive Answers)

Q21: Tell me about yourself.

A:
I’m a MERN Stack Developer with an interest in AI-driven applications.
I enjoy building practical tools like this AI Code Reviewer.
I’m strong in JavaScript, React, Node.js, and integrating APIs.
I focus on clean code, problem-solving, and continuous learning.

Q22: Why should we hire you?

A:
I have strong technical skills, hands-on project experience, and the ability to learn fast.
I can contribute to frontend, backend, and AI integration tasks.

A:
I reproduce the issue, isolate the cause, check logs, and fix step-by-step.
I stay calm and logical during debugging.

✅ 4. HR General Questions (Positive Answers)

Q21: Tell me about yourself.

A:
I'm a MERN Stack Developer with an interest in AI-driven applications.
I enjoy building practical tools like this AI Code Reviewer.
I'm strong in JavaScript, React, Node.js, and integrating APIs.
I focus on clean code, problem-solving, and continuous learning.

Q22: Why should we hire you?

A:
I have strong technical skills, hands-on project experience, and the ability to learn fast.
I can contribute to frontend, backend, and AI integration tasks.
I'm reliable, positive, and deliver high-quality work.

Q23: What are your strengths?

A:

- Quick learner
- Strong problem-solving
- Clean coding practices
- Good communication
- Ability to work under pressure

Q24: What are your areas of improvement?

A:
I am improving my backend optimization and system design skills.
I practice regularly by building scalable projects.

Q25: Where do you see yourself in 2 years?

A:
I see myself as a skilled full-stack developer working on AI-based applications and handling more responsibilities.

✅ 5. Scenario-Based Questions (Positive Answers)

Q26: What if the AI API fails during a user request?

A:
I would implement proper error handling, retries, and fallback messaging.
The user would see a meaningful message instead of an error.

Q27: What would you do if your teammate disagrees with your implementation?

I practice regularly by building scalable projects.

Q25: Where do you see yourself in 2 years?

A:
I see myself as a skilled full-stack developer working on AI-based applications and handling more responsibilities.

✅ 5. Scenario-Based Questions (Positive Answers)

Q26: What if the AI API fails during a user request?

A:
I would implement proper error handling, retries, and fallback messaging.
The user would see a meaningful message instead of an error.

Q27: What would you do if your teammate disagrees with your implementation?

A:
I'd listen to their perspective, compare both approaches, and choose the best solution based on logic, not ego.
If needed, I'm open to adjusting my approach.

Q28: What if your project deadline is close and a critical bug appears?

A:
I stay calm, prioritize the bug, fix it systematically, and communicate with the team if extra time is needed.

Q29: How would you improve the performance of your application?

A:

- Add API caching
- Minimize frontend re-renders
- Optimize backend routes
- Use environment variables and configuration management

Q30: If AI gives wrong or harmful suggestions, how would you handle it?

A:
I'd sanitize inputs, validate responses, and set boundaries in the prompt to ensure safe and useful output.

⭐ If you want, I can also prepare for you:

✔ HR Round full script
✔ Final-year project viva Q&A
✔ Resume points (3-line/5-line versions)
✔ A short video-style project explanation

Just tell me!

Give stepwise explanation of this project what is the process of building

Step-by-step build process — *AI-Powered Code Reviewer (MERN + Google GenAI)*

Below is a practical, sequential roadmap you can follow (and explain in interviews) to build the project from scratch. Each step includes the why, what to do, and example snippets/commands where useful.

---

1. Plan & design

Why: define scope, data flow and UX before coding.
Do:

- Decide core features: paste code, choose language, request review, show review (Markdown), history (optional).
- Draw a simple flow: Frontend → Backend → Google GenAI → Backend → Frontend.
- Choose stack: React (Vite), Node + Express, MongoDB (optional).
- Create repo and initial README.

Example:

```csharp
git init
git remote add origin <your-git-url>
```

---

2. Initialize backend (Node + Express)

Why: backend handles requests, prompts the AI, and secures API keys.
Do:

- Create project folder `server/` and init npm.

```bash
mkdir server && cd server
npm init -y
npm i express cors axios dotenv
```

- Create `server/index.js` (or `app.js`) and basic Express server:

```js
// server/index.js
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors({ origin: process.env.CLIENT_ORIGIN }));
app.use(express.json());

app.post('/api/review', async (req, res) => {
  const { code, language } = req.body;
  // send to AI later
  res.json({ ok: true, message: 'placeholder' });
});

const PORT = process.env.PORT || 4000;
```

- Create `server/index.js` (or `app.js`) and basic Express server:

```js
// server/index.js
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors({ origin: process.env.CLIENT_ORIGIN }));
app.use(express.json());

app.post('/api/review', async (req, res) => {
  const { code, language } = req.body;
  // send to AI later
  res.json({ ok: true, message: 'placeholder' });
});

const PORT = process.env.PORT || 4000;
app.listen(PORT, () => console.log(`Server running on ${PORT}`));
```

- Add `.env` for keys (gitignore it):

```ini
PORT=4000
CLIENT_ORIGIN=http://localhost:5173
GEN_AI_API_KEY=sk_xxx
```

3. Design prompt structure & AI integration

Why: prompt engineering determines the quality of reviews. Keep it structured.

Do:

- Create function that formats prompt with context (language, goal, constraints).
- Use the Google Generative AI API (or chosen provider) via Axios/fetch.

Example prompt pattern:

```scss
"You are a senior software engineer. Review the following <language> code.
```

Example axios call (pseudo):

```js
const axios = require('axios');

async function callGenAI(prompt) {
  const resp = await axios.post('https://api.generative.google/v1/models/x
    prompt
  }, {
    headers: { Authorization: `Bearer ${process.env.GEN_AI_API_KEY}` }
  });
  return resp.data;
}
```

4. Implement backend route logic

Why: transform request → prompt → AI → response → sanitized return.

Do:

4. Implement backend route logic

Why: transform request → prompt → AI → response → sanitized return.

Do:

- Validate input size and language.
- Build final prompt + send to API.
- Post-process AI response (optional: parse headings, trim, ensure safe content).
- Return Markdown-ready string to client.

Example route:

```js
app.post('/api/review', async (req, res) => {
  try {
    const { code, language='javascript' } = req.body;
    if (!code) return res.status(400).json({ error: 'No code provided' });

    const prompt = `Review this ${language} code: \n\n${code}\n\nRespond a
    const aiResult = await callGenAI(prompt);
    const aiText = aiResult?.output?.[0]?.content ?? 'No response';
    res.json({ review: aiText });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'AI request failed' });
  }
});
```

5. Initialize frontend (React + Vite)

Why: create interactive UI quickly.

Do:

```sql
npm create vite@latest client --template react
cd client
npm install axios prismjs react-markdown
```

Project structure suggestion:

```css
client/
  src/
    App.jsx
    components/
      CodeEditor.jsx
      ReviewDisplay.jsx
    services/
      api.js
```

6. Build code input UI (syntax highlighting)

Why: good UX for developers. Prism.js or a lightweight editor works well.

Do:

- Simple textarea + Prism to display highlighted preview OR integrate a code editor like CodeMirror/Monaco (optional).
- Add language selector.

6. Build code input UI (syntax highlighting)

Why: good UX for developers. Prism.js or a lightweight editor works well.

Do:

- Simple textarea + Prism to display highlighted preview OR integrate a code editor like CodeMirror/Monaco (optional).
- Add language selector.

Simple component example:

jsx Copy code

```jsx
// CodeEditor.jsx (simplified)
import React, { useState } from 'react';
export default function CodeEditor({ onSubmit }) {
  const [code, setCode] = useState('');
  const [language, setLanguage] = useState('javascript');
  return (
    <div>
      <select value={language} onChange={e=>setLanguage(e.target.value)}>
        <option>javascript</option>
        <option>python</option>
        ...
      </select>
      <textarea value={code} onChange={e=>setCode(e.target.value)} rows={1
      <button onClick={()=>onSubmit({ code, language })}>Review</button>
    </div>
  );
}
```

7. Connect frontend → backend (Axios)

Why: send code safely as POST.

Do:

- Create `services/api.js`:

js Copy code

```js
import axios from 'axios';
export const requestReview = (payload) => axios.post('/api/review', payloa
```

- In `App.jsx`, call and handle loading, error states:

jsx Copy code

```jsx
const handleReview = async ({ code, language }) => {
  setLoading(true);
  try {
    const { data } = await requestReview({ code, language });
    setReview(data.review);
  } catch (e) { setError('Failed'); }
  setLoading(false);
};
```

8. Render AI response (Markdown + styling)

Why: GenAI outputs are text with code blocks and headings — Markdown renderer makes it clean.

Do:

8. Render AI response (Markdown + styling)

Why: GenAI outputs are text with code blocks and headings — Markdown renderer makes it clean.

Do:

- Install `react-markdown` and show `review`:

```jsx
import ReactMarkdown from 'react-markdown';
<ReactMarkdown children={reviewText} />
```

- Style code blocks with Prism highlighting inside `react-markdown` renderers if you want highlighting in results.

9. Add CORS, security & rate limiting

Why: protect API keys and prevent abuse.

Do:

- Keep API key on server only (`.env`).
- Limit request size: `express.json({ limit: '100kb' })` or appropriate.
- Add rate limiter: `express-rate-limit` to avoid spam to GenAI.
- Validate and sanitize user input (avoid prompt injection).

10. Add persistence (Optional)

Why: save user history, comparisons, analytics.

Do:

- Add MongoDB + Mongoose:

```css
npm i mongoose
```

- Create `Review` model and save review results with metadata (user, language, timestamp).

11. Test thoroughly

Why: ensure reliability & robust behavior with edge cases.

Do tests:

- Unit test backend functions (prompt builder, response parser).
- Manual UI tests: large code snippets, empty input, invalid language.
- Test network errors & timeouts.
- Confirm AI output formatting and sanitization.

Checklist:

- [ ] Valid input returns review
- [ ] Long code handled (or gracefully declined)
- [ ] Error messages are user-friendly
- [ ] No API key leaked in responses or client code
- [ ] Rate limit works

12. Logging & monitoring

Why: debug production issues

12. Logging & monitoring

Why: debug production issues.
Do:

- Add basic logging (winston or morgan).
- Track errors and latency (Sentry, LogDNA, or simple logs).
- Monitor AI usage and cost.

13. Deploy (production)

Why: make app available.
Do:

- Frontend: Vercel / Netlify (build `client` and deploy).
- Backend: Render / Heroku / Railway (set environment variables).
- Ensure `CLIENT_ORIGIN` matches deployed frontend URL and enable HTTPS.
- Set process manager if needed (PM2).

Example:

- Set `GEN_AI_API_KEY` in server host env vars.
- Configure CORS to allow the deployed frontend origin.

14. Improvements & features roadmap

Why: show growth path in interviews.
Ideas:

- Authentication + user profiles and saved reviews.
- Multiple AI modes: quick tips, in-depth refactor, security audit.
- Linting integration and real-time inline suggestions.
- WebSocket for streaming AI responses.
- Compare before/after code diff view.
- Team collaboration (share reviews).
- Cost optimization: caching repeated reviews for identical code.

15. Demo & presentation tips (for interviews)

Why: help you confidently demo the project.
Do:

- Prepare a short script: problem → solution → demo flow (paste code, click review, explain AI output).
- Highlight one tricky bug the AI found (real example).
- Mention security: API keys never in client, rate limiting, input validation.
- Discuss what you'd do next (scaling, auth, more languages).

Short demo script:

1. "I'll paste this function — it has an edge-case bug."
2. Click Review — explain the loading spinner.
3. Show AI result: summary, bug found, suggested fix.
4. Explain how backend formatted prompt & sanitized response.

16. Example file structure

bash Copy code

4. Explain how backend formatted prompt & sanitized response.

16. Example file structure

bash Copy code

```
/project-root
  /client
    src/
      components/
        CodeEditor.jsx
        ReviewDisplay.jsx
      services/api.js
      App.jsx
  /server
    index.js
    aiClient.js
    routes/review.js
    models/Review.js   (optional)
  .env
  README.md
```

17. Security & ethical considerations

Why: important for production and interview discussions.

Do:

- Filter out sensitive code (API keys) in user submissions with warnings.
- Avoid blindly applying AI suggestions — present as suggestions.
- Include user confirmation before applying code changes.
- Log abusive or harmful prompts and implement safeguards.

18. CI/CD & automation (optional)

Why: professionalizes the project.

Do:

- Add GitHub Actions to run tests and lint on push.
- Auto-deploy to hosting on merge to `main`.

19. Final polish & README

Why: makes the repo interview-ready.

Do:

- Add clear README with features, architecture diagram, setup steps, env vars, and demo GIF.
- Add short “How it works” section with endpoints and prompt example.

Quick reference snippets

Backend prompt builder (example):

js Copy code

```
function buildPrompt(code, language) {
  return `
You are a senior ${language} developer.
Task: Review the code below and provide:
```

Do:

- Filter out sensitive code (API keys) in user submissions with warnings.
- Avoid blindly applying AI suggestions — present as suggestions.
- Include user confirmation before applying code changes.
- Log abusive or harmful prompts and implement safeguards.

---

18. CI/CD & automation (optional)

Why: professionalizes the project.

Do:

- Add GitHub Actions to run tests and lint on push.
- Auto-deploy to hosting on merge to `main`.

---

19. Final polish & README

Why: makes the repo interview-ready.

Do:

- Add clear README with features, architecture diagram, setup steps, env vars, and demo GIF.
- Add short “How it works” section with endpoints and prompt example.

Quick reference snippets

Backend prompt builder (example):

```js
function buildPrompt(code, language) {
  return `
You are a senior ${language} developer.
Task: Review the code below and provide:
1) Short summary
2) Possible bugs
3) Performance/security concerns
4) Refactor suggestions with code
5) Best practices

Code:
\`\`\`${language}
${code}
\`\`\`
Respond in Markdown.
`;
}
```

Frontend Axios call example:

```js
import axios from 'axios';
export const requestReview = (payload) =>
  axios.post('http://localhost:4000/api/review', payload, { timeout: 60000
```